

MINI PROJECT SYNOPSIS

AI STOCK INSIGHTS PORTAL

A Web-Based Technical-Analysis Dashboard for Indian Equity Markets

**SUBMITTED FOR THE PARTIAL FULFILLMENT OF
THE B.TECH. (BRANCH) II YEAR III SEMESTER COURSE**

**INTERNSHIP/MINI PROJECT – BCC 351
DRONACHARYA GROUP OF INSTITUTIONS**



BY

Rohit(2502301530051)

Shubh(2502301530059)

Utkarsh(2502301530069)

Affiliated to

**Dr A P J ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW
ACADEMIC SESSION 2025-26**

AI STOCK INSIGHTS PORTAL

Content

The AI Stock Insights Portal is a Streamlit web application that makes technical analysis of Indian National Stock Exchange (NSE) equities accessible through a browser-based dashboard. It retrieves up to one year of historical market data for a selected ticker, calculates technical indicators, and produces an explainable BUY, SELL, or HOLD signal. The portal supports market overview, single-stock analysis, and two-stock comparison. It is designed as an educational decision-support tool and not as investment advice or a price-prediction system.

Project Type	Technology
Mini Project	Python, Streamlit, yfinance, pandas, pandas-ta, Plotly

Objective

- Fetch recent historical market data for NSE-listed stocks entered by the user.
- Calculate RSI (14), SMA (50), and SMA (200) from closing-price data.
- Classify the current technical condition as BUY, SELL, or HOLD using transparent rules.
- Present a market overview of selected popular Indian equities.
- Display detailed candlestick analysis and enable comparison between two stocks.
- Provide a simple session-based login and registration interface for dashboard navigation.

About the Project

Equity-market users often need to interpret changing price movements, momentum, and trends. Raw price data can be difficult to interpret quickly, especially for learners. The portal consolidates market-data retrieval, indicator computation, signal generation, tabular summaries, and charting in a single interface. It uses common NSE ticker suffixes such as RELIANCE.NS and TCS.NS and includes a curated stock list for an at-a-glance view.

The implemented AI signal is deterministic and rule-based. It combines the latest 14-period RSI with the relationship between closing price and the 50-day moving average. It does not train a machine-learning model, forecast exact future prices, incorporate company fundamentals, or place trades.

Functional Requirements

Feature	Inputs / Processing	Output
Login / registration	Username and password fields stored in session state	Authenticated dashboard access
Market overview	Ten predefined NSE tickers; per-ticker analysis	Summary table with price, RSI, SMA, signal, and reason
Single-stock analysis	User ticker symbol	Key metrics, explanation, and candlestick chart
Stock comparison	Two ticker symbols	Side-by-side comparison
Refresh / validation	Live data request; minimum 50 records checked	Updated view or descriptive error

Project Flow / Methodology

The application follows a request-response workflow. A user signs in, selects an analysis mode, and supplies one or two ticker symbols where required. The shared analysis function normalizes the symbol, retrieves historical data, computes indicators, evaluates signal rules, and returns metrics or an error. The user-interface module then presents the result through tables, metric cards, explanatory text, or an interactive chart.

APPLICATION WORKFLOW

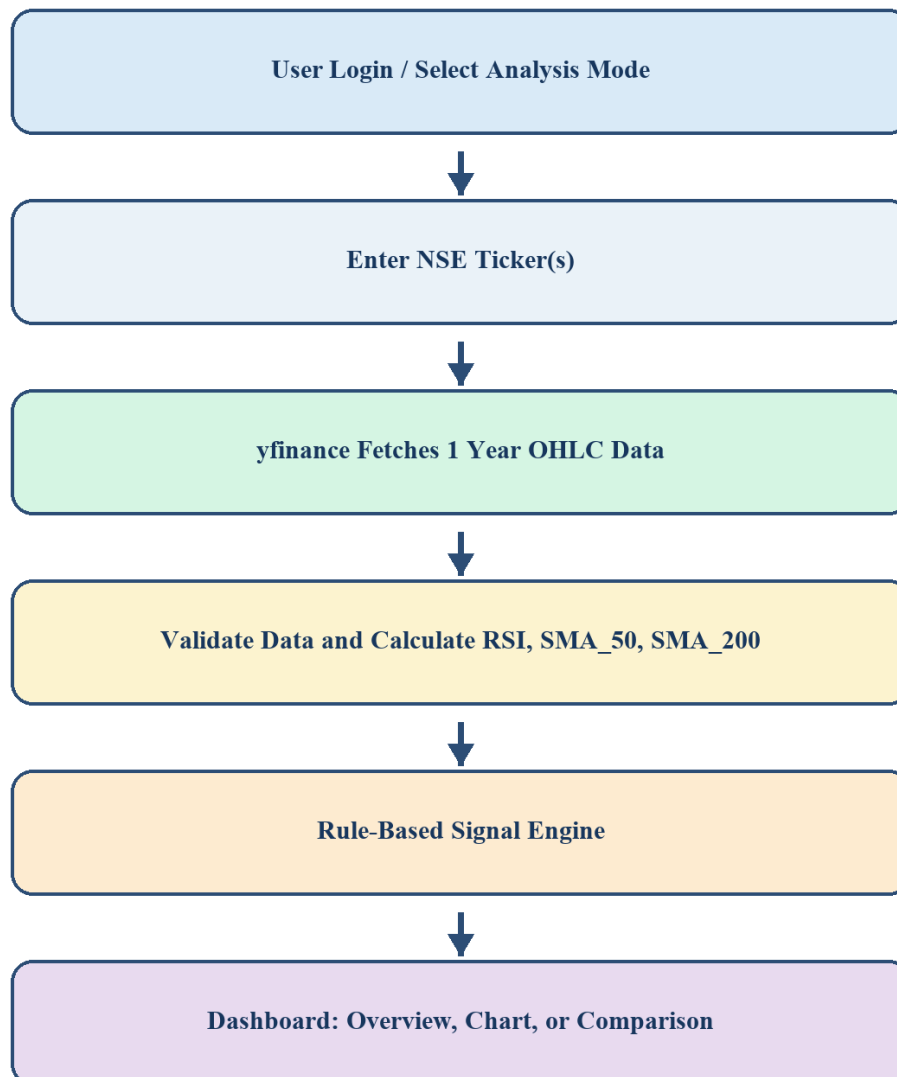


Figure 1. Application workflow of the AI Stock Insights Portal.

- Presentation layer: Streamlit pages, tabs, sidebar, forms, metrics, and tables.
- Data-access layer: yfinance retrieves one year of OHLC price history.
- Analytics layer: pandas-ta calculates RSI and simple moving averages.
- Decision layer: conditional logic maps market conditions to signal and confidence.
- Visualisation layer: Plotly renders a candlestick chart and 50-day SMA line.

Signal Logic

For each ticker, the program converts input to uppercase, downloads one year of historical OHLC data, and checks that at least 50 records are available. It calculates RSI using a 14-period lookback, SMA_50 from the closing-price series, and SMA_200 for longer-horizon context. The most recent values determine the signal, confidence, and explanatory reasons.

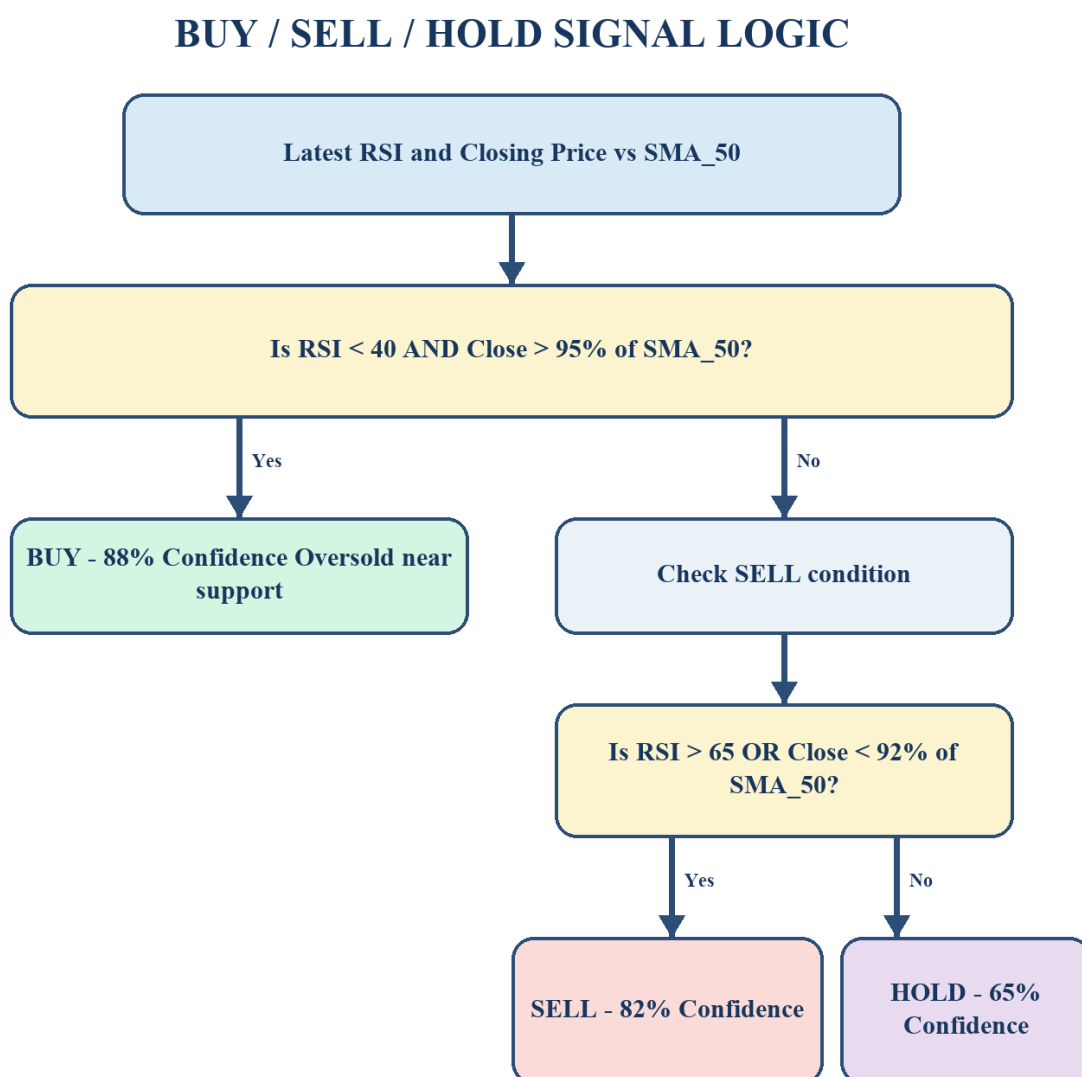


Figure 2. Rule-based BUY, SELL, and HOLD signal logic.

Condition	Result	Interpretation
RSI < 40 and Close > 95% of SMA_50	BUY; confidence 88%	Oversold condition near moving-average support
RSI > 65 OR Close < 92% of SMA_50	SELL; confidence 82%	Overbought condition or notable break below SMA support
All other cases	HOLD; confidence 65%	Consolidation or no strong trigger

Technology Stack

- Python for core programming logic.
- Streamlit for the interactive web interface and session-state management.
- yfinance for historical market-data retrieval.
- pandas for data-frame handling and display-table construction.
- pandas-ta for RSI and simple moving-average calculation.
- Plotly Graph Objects for interactive candlestick and moving-average charting.

Expected Outcome

The completed project provides a practical dashboard through which users can inspect an NSE stock's latest technical state, understand the basis for a generated signal, and compare two stocks under consistent rules. It demonstrates integration of real-world data APIs, analytical libraries, interactive visualisation, and stateful web-user-interface design in a compact Python project.

Limitations and Future Enhancements

- The signal engine is rule-based; labels and confidence percentages are fixed heuristic outputs rather than calibrated predictions.
- Live-data quality, availability, and delay depend on the external yfinance source.
- Authentication is session-only and does not provide persistent accounts or secure credential storage.
- Future work can add secure database authentication, caching, additional indicators, watchlists, alerts, backtesting, and explainable machine-learning models.

References

- Streamlit Documentation. <https://docs.streamlit.io/>
- yfinance Documentation. <https://ranaroussi.github.io/yfinance/>
- pandas Documentation. <https://pandas.pydata.org/docs/>
- pandas-ta Documentation. <https://github.com/twopirllc/pandas-ta>
- Plotly Python Documentation. <https://plotly.com/python/>

Disclaimer: This project is intended for learning and informational use. Its outputs should not be treated as personalised investment advice or a guarantee of future market performance.